Spring Boot Assessment – Student Management System

Problem Statement

Develop a **Student Management System REST API** using **Spring Boot**.
The application should allow administrators to manage student records and allow students to view their own details securely. The system must also support file uploads, pagination, caching, and monitoring.

Students must implement the application using:

- Spring Boot
- Spring Data JPA
- REST APIs
- Pagination and Sorting
- Spring Cache
- Multipart File Upload
- Spring Security (Basic Authentication)
- Password Encryption
- Role-Based Authorization
- Method-Level Security (@PreAuthorize, @PostAuthorize)
- CORS Configuration
- Spring Boot Actuator

---

1. Student Entity

Create a **Student entity** with the following fields:

- id
- name
- email
- course
- marks
- profileImage
- assignmentFile

The **profile image and assignment file must be stored directly in the database** using binary format. Use byte[] data type and mark them as **large objects** so that the database can store large binary data.

---

2. REST APIs

Develop REST APIs to perform the following operations:

- Create a new student record
- Retrieve student details by ID
- Retrieve all students
- Update student details
- Delete a student record

The APIs should follow proper **RESTful design principles**.

---

3. Pagination and Sorting

The API that retrieves all students must support **pagination and sorting** so that results can be retrieved in smaller pages and ordered based on a specific field.

Students should implement pagination using **Spring Data JPA pageable features**.

---

4. File Upload

The application must allow file uploads using **multipart requests**.

Two types of uploads must be supported:

1. **Student Profile Image Upload**
2. **Student Assignment File Upload**

When a file is uploaded:

- Accept the file using multipart request.
- Convert the uploaded file to binary format.
- Store the file data inside the database in the respective field of the student entity.

---

5. File Download

Provide APIs that allow users to download:

- Student profile image
- Student assignment file

The stored binary data must be returned as a response so that the client can download the file.

---

6. Spring Cache

Enable **Spring Cache** in the application.

Cache the result of retrieving student details by ID so that repeated requests for the same student do not always hit the database.

---

7. Spring Security (Basic Authentication)

Secure the application using **Spring Security Basic Authentication**.

User credentials must be stored in the **database**.
Passwords must be **encrypted before storing**.

Use role-based authentication with at least two roles:

- ADMIN
- USER

---

8. Role-Based Authorization

Configure **URL-level security** using Spring Security.

- Only administrators should be able to create, update, or delete student records.
- Both administrators and users should be able to retrieve student records.

Authorization rules must be implemented using **request matchers with role-based access control**.

---

9. Method-Level Security

Enable **method-level security** in the application.

Use both:

- @PreAuthorize
- @PostAuthorize

@PreAuthorize should be used to restrict access before executing certain service methods.

@PostAuthorize should be used to validate the returned result and ensure that a logged-in user can only access their own student information based on the authentication details.

---

10. CORS Configuration

Configure **Cross-Origin Resource Sharing (CORS)** so that the API can be accessed from a frontend application running on another origin.

---

11. Spring Boot Actuator

Enable **Spring Boot Actuator** in the project.

Expose important monitoring endpoints that allow checking:

- Application health
- Application information
- Application metrics

Expected Project Structure

Students should organize the application using standard **Spring Boot layered architecture**, including:

- Entity layer
- Repository layer
- Service layer
- Controller layer
- Security configuration
- File handling logic
- Caching configuration

Objective of the Assessment

This assignment is designed to evaluate the student's understanding of:

- Spring Boot application development
- JPA and database interaction
- REST API design
- File handling in Spring Boot
- Security implementation with role-based authorization
- Method-level authorization
- Performance optimization using caching
- Pagination and sorting
- Monitoring using Actuator